实验四：Benchmark 评估与 HBirdv2 E203 流水线优化（DDR200T FPGA）

一、实验目的

1．理解流水线结构与性能指标的关系

能基于处理器流水线（取指/译码执行回写/访存等）解释 CPI、IPC、吞吐率等指标，并能将“冒险——代价——性能损失”建立因果关系。（E203 属于以“取指”为第一阶段、译码/执行/回写为第二阶段的典型两级流水线结构，并存在 LSU 访存通路。

2．掌握 benchmark 的构建、运行与性能数据采集方法

能在 HBirdv2 E203 SoC 上运行一组可重复的 benchmark（建议采用 CoreMark），并用硬件计数器采集周期数与 Retire 指令数，计算 CPI/IPC。

3．面向瓶颈进行微结构优化并定量评估效果

在 HBirdv2/E203 开源 RTL 上实施至少一种“可解释的”流水线/结构优化（如减少 load-use 停顿、降低控制相关代价、或降低多周期单元阻塞等），并以 benchmark 数据量化收益（speedup、CPI 变化、吞吐率变化）。

二、预习要求

1．教材阅读（Stallings 第 10 版，建议范围）

（1）流水线基本概念与吞吐率/加速比

（2）CPI 分解（理想 CPI+停顿项），结构/数据/控制冒险及常见处理（stall、forwarding、分支预测等）

2．平台与内核资料

（1）阅读 HBirdv2 E203 Core 文档：重点了解“2-Stage Pipeline”“可配置项（config.v）”“MCYCLE/MINSTRET 支持”等。

（2）了解 DDR200T 板卡基本构成（FPGA 子系统+MCU 子系统、XC7A200T、JTAG/调试器结

构、电源规格）。

3. 性能计数器与测量基础

（1）预习 RISC-V 特权架构中 mcycle 与 minstret 的定义与用途（周期数/Retire 指令数）。

4. Benchmark 认知

（1）CoreMark；EEMBC 的 MCU/CPU 基准，常用于嵌入式整数性能对比。

三、实验环境与准备

1. 硬件

（1）芯来科技 Nulcei DDR200T FPGA Evaluation Kit（12V 外部直流供电；FPGA：Xilinx XC7A200T-2；板载存储 Flash/DDR/eMMC/EEPROM；HBird Debugger 接口；MCU 子系统 GD32VF103 等）。

（2）HBird Debugger（或等效可用的板载/外置调试方案，需支持 JTAG 与串口输出）。

2. 软件与代码仓库

（1）RTL/SoC：HBirdv2 E203 开源工程

https://github.com/riscv-mcu/e203_hbirdv2

（2）SDK：HBird SDK（Makefile 工作流，支持 ddr200t + hbirdv2 + e203 组合）

https://github.com/riscv-mcu/hbird-sdk

（3）工具链：Nuclei RISC-V GCC、OpenOCD（按 HBird SDK Quick Start 配置）

3. 基准基线

（1）使用 NucleiStudio 的 CoreMark 例程作为 baseline benchmark 示范

（2）必须符合 EEMBC CoreMark 的 run/report 规则（即执行结果含 CRC 验证及 CoreMark 分数；且 single-thread 运行）

四、实验内容与任务概览

整体实验分为四个阶段：

A） 基线配置与 CoreMark 跑通验证

B） 性能测量框架搭建与指标采集

C） 基准运行与性能数据采集/分析

D） 微结构优化设计与 Performance 对比

阶段 A/B/C 是必做项，阶段 D 是必做 1 个方向，结果需量化对比。

A. 基线环境与 CoreMark 基准跑通

a) 目标：确认基线系统（HBirdv2/E203 + DDR200T FPGA + CoreMark 示例工程）能稳定运行，并输出符合 CoreMark benchmark 的标准结果。CoreMark 是 EEMBC 定义的嵌入式处理器基准，用于评估处理器整数运算、控制流和基本数据结构操作等核心性能。其输出包括单一基准分数，并且需满足 CRC 自检规则以保证正确性。

b) 任务要求：

   i. 成功下载、编译、并在 DDR200T FPGA 上运行 CoreMark 示例工程。

   ii. 串口日志必须包含 CoreMark 官方输出关键项：

      1. Total ticks / Total time

      2. Iterations/Sec 或 CoreMark score

      3. CRC self- check 成功输出（如 Correct operation validated）。

c) 输出产物;

   i. Baseline CoreMark 运行结果截图或日志文本。

   ii. 名义的 CoreMark 得分（如 Iterations/Sec 或 CoreMark/MHz）。

B. 性能测量框架搭建与指标采集

a) 目标：在现有 CoreMark 运行基础上，搭建更精细的性能测量框架，使得可以从微结构视角量化处理器性能，包括 CPI（Cycles Per Instruction）、IPC（Instructions Per Cycle）。这些指标能够反映流水线停顿、指令冒险等性能瓶颈。

b) 技术说明：

i. CoreMark 的输出主要是高层的时间/迭代统计，但不能直接揭示处理器内部行为。

ii. RISC-V 提供 mcycle / minstret 性能计数器，可分别统计 CPU 周期和退休指令数量，进而计算 CPI/IPC。

iii. CPI 和 IPC 是衡量流水线性能和效率的核心指标：

1. IPC = Instret / Cycles
2. CPI = Cycles / Instret

c) 任务要求：

i. 在 CoreMark 基线执行区域之外实现性能计数器采样（可在客户端/SDK 侧完成）。

ii. 对每次基准运行记录多个性能数据点，包括周期数、Retire 指令数及内存访问模式（若可用）。

iii. 提供基线 CPI/IPC 计算过程和结果。

d) 输出产物：

i. 包含周期数与 Retire 指令数的性能数据表。

ii. 计算出的 Baseline CPI/IPC 值。

C. 基准运行与性能数据分析

a) 目标：系统性运行 CoreMark 基准，并结合性能计数器数据，分析基线处理器在不同工作负载下的性能特征，从而识别可能的性能瓶颈。这一步是优化前理解体系结构行为的关键。

b) 技术说明：

i. CoreMark 由若干典型算法组合而成，包括链表操作、矩阵运算、状态机等，能综合测试整数运算、控制逻辑和部分内存访问行为。

ii. 通过对比不同任务阶段的 CPI/IPC，以及结合 CoreMark 输出的迭代数和时间，可以推断主要性能限制（如 load- use 停顿、多次分支跳转停顿等）。

c) 任务要求：

i. 对基线工程进行多次独立运行，且每次结果需满足 CoreMark 的稳定性要求（如运行 ≥ 10 秒有意义统计区间）。

ii. 统计和比较多次运行的平均值、标准差等统计指标。

iii. 结合代码特征（指令种类分布、关键循环热点）解释 KPI 结果，例如为什么 CPI > 理想值、IPC < 理想值。

d) 输出产物：

i. 多次 CoreMark 运行结果的统计表。

ii. 对 CPI/IPC 与指令分布的分析报告片段。

iii. 定性和定量结合的性能瓶颈识别结论。

D. 流水线微结构优化与效果对比

a) 目标：在理解基线性能行为的基础上，有针对性地修改流水线相关微结构（例如冒险 forwarding、分支处理策略、load- use penalty 缓解等），并量化评估这些优化对性能的影响。优化设计应围绕让 CPI 下降、IPC 上升或总体迭代率提升展开。

b) 技术说明：

i. 流水线性能损失往往来自冒险（结构、数据、控制），优化通常围绕减少冒险停顿、降低分支误判代价、增强旁路功能等方向展开。

ii. 由于 CoreMark 基准的混合性质，它能在统计层面体现这些改动的累积性能变化。

iii. 优化后的对比需保持所有其他条件（工具链、编译 flags、CoreMark 配置、输入参数）一致，以保证对比的有效性和公平性。

c) 任务要求：

i. 选取至少一种流水线相关的微结构优化方向（见附录）。

ii. 在 RTL 或相关配置文件中实现改动。

iii. 重新运行 CoreMark 基准，并重新采集性能指标。

iv. 以“基线 vs 优化”对比表格形式呈现核心数据（Cycles、Instret、CPI、IPC、CoreMark/MHz）。

v. 分析改动对性能的贡献，以及潜在的代价（如复杂度增加、时序压力等）。

d) 输出产物：

i. 优化后 CoreMark 结果与性能计数器统计表。

ii. 对比分析文档：改动设计意图、性能提升量化结果与解释。

E. 总结性输出要求

a) 为保证学生能清晰展示实验成果，每个任务模块需要生成成果文档或表格，包括但不限于

结果表格格式（建议）：

项目	Baseline	Optimized
CoreMark Iterations/Sec		
Total ticks		
Cycle count		
Instruction Retired		
CPI		
IPC		
CoreMark/MHz		

五、实验步骤

1. 克隆 HBirdv2 E203 RTL 代码库（包含完整 SoC 模板）：

```
git clone https://github.com/riscv-mcu/e203_hbirdv2.git
```

2. 获取 hbird- SDK 和 Nuclei RISC- V 工具链（用于编译和下载 CoreMark）：

```
git clone https://github.com/riscv-mcu/hbird-sdk.git
```

3. 按官方文档环境搭建指南，配置 Nuclei RISC- V GCC、OpenOCD 等工具链，设置环境变量（如 NUCLEI_TOOL_ROOT）。

4. 初始化 FPGA 环境

（1）连接 DDR200T FPGA 板卡并确认上电；连接调试器、串口线；

（2）打开终端程序（如 Tera Term / minicom），设置串口波特率（通常 115200）。

5. Baseline 工程编译运行

（1）在 hbird-SDK 中进入 CoreMark 的示例工程目录；

（2）修改 Makefile 使用固定的 ITERATIONS（如 2000–4000，以确保 ≥ 10 秒运行）；

（3）编译并下载至 FPGA；

（4）观察串口输出，确认至少包括以下行：Total ticks、Iterations/Sec、CoreMark Size；CRC

Pass/Fail

6．在 CoreMark 中加入性能计数：为了测量 CPI/IPC（性能计数器数据），建议在 CoreMark timed region 前后加上对 RISC- V CSR 计数器的读操作：

（1）在 start_time(); ... stop_time(); 前后插入读取 mcycle / minstret 的代码；

（2）保证计数器已启用（E203 默认支持 mcycle / minstret CSR）。

7．记录测量数据：在串口输出或日志文件中记录如下内容：

（1）周期计数（mcycle）

（2）退休指令计数（minstret）

（3）计算得到的 CPI/IPC 数值

（4）CoreMark 官方得分（Iterations/Sec / CoreMark/MHz）

8．基线性能数据采集与分析

（1）重复执行 CoreMark 至少 5 次，取平均值用于性能分析；

（2）在实验日志中写出表格内容，如下：

指标	单位	运行 1	运行 2	...	均值
Total ticks	ticks				
mcycle	cycles				
minstret	retired				
CPI	cycles/inst				
IPC	inst/cycle				
Iterations/Sec	/s				

（3）结合 CoreMark 中各子 benchmark 的行为分析性能瓶颈，例如 load → use 冒险数量、branch 密度等。

9．RTL 逻辑理解与优化路径确定：在做任何 RTL 修改前，必须理解 E203 的 pipeline 结构与模块分布：

（1）Fetch（IFU）

（2）Decode / Execute / Write Back（EXU）

（3）Load/Store Unit（LSU）

（4）打开 e203_hbirdv2/rtl/e203/core 目录，核心模块大致如下：

```
e203_cpu_top
├── e203_core
│   ├── e203_ifu
```

```
│   ├── e203_exu
│   ├── e203_lsu
│   ├── e203_biu
│   └── ...
```

这些模块分别实现不同 pipeline stage 的逻辑。

10．具体 RTL 修改与优化实现（示例方向）

（1）示例：具体 RTL 修改与优化实现（示例方向）

（2）修改位置与方向：

在 LSU 与 EXU 之间增加 forwarding path：在 e203_lsu 中添加信号输出数据到执行前级；

在 EXU hazard detection logic（位于 e203_exu）中增加检测和数据前推路径。

修改 hazard detect unit 以避免某些简单 load → use 停顿，并通过 forwarding 提前提供数据。

（3）操作步骤（示例）：

修改 e203_exu 中与 operand mux 控制相关的信号选择逻辑，将来自 LSU 的数据通过新的 forwarding mux 传入执行单元；

在 config.v 中（RTL config），添加 enable forwarding 的宏定义（若可配置）。

（4）验证：

加载新的 RTL；

仿真测试案例检查 load → use 相关的结果保持正确；

下载至 FPGA 跑 CoreMark 并观察 CPI/IPC 是否下降。

11．RTL 变更后的仿真与验证

（1）用仿真工具（如 Vivado iSim / Icarus / ModelSim）模拟修改后的核心；

（2）用简单单元测试检测 correctness；

（3）再用 CoreMark 运行输出结果或验证指令行为与 baseline 保持一致。

12．优化版下载与性能采集

（1）生成 bitstream/MCS；

（2）下载至 FPGA；

（3）收集优化版的：

CoreMark 输出；

周期数/退休指令（mcycle / minstret）；

计算优化后的 CPI / IPC。

13．对比与报告：按照第六章的报告要求，将 baseline 与优化版的指标放在同一表格并进行分析。

六、实验记录与报告要求

1．基线配置与源代码版本

（1）工具链说明：完整报告 CoreMark 运行规则要求的编译器版本与 flags（C - Compiler version and flags）。

（2）CoreMark 配置参数：报告 Iterations、Memory 位置（如 Heap / Stack / Static）、MULTITHREAD 配置等所有参数。

（3）运行条件：FPGA 时钟设置/主频、上电模式等。

2．必要的性能数据量化指标

（1）报告必须按统一量纲记录以下指标：

指标	Baseline	优化版	单位	说明
CoreMark Size			(字节)	CoreMark 缓冲区规模
Iterations			次	核心迭代次数
Total ticks			tick	CoreMark 内部衡量时间单位
Total time (secs)			s	CoreMark 估算的运行时间
Iterations/Sec			/s	CoreMark 官方得分基础
Compiler version			文本	完整编译链标识
Compiler flags			文本	必须报告以符合 run rule
Cycle count（mcycle）			周期	必须通过 CSR 读出
Instruction Retired（minstret）			指令数	同上
CPI			周期/指令	= cycle/instret
IPC			指令/周期	= instret/cycle
CoreMark/MHz			得分	归一化性能指标
CRC Pass/Fail			布尔	CoreMark self- check 成功指示

3．多次试验与统计

（1）每个配置至少执行 3 次独立运行，并写出平均值、标准差等统计量，以减少单次波动干扰（例如串口延迟或环境因素）。

（2）对比表中要体现 variance，以说明实验可重复性。

4．性能计数器采集说明：

因为 CoreMark 自身输出不包含 CPI/IPC，为满足实验学习目标，报告中必须包含性能计数器结果的采集说明：

（1）计数器源码片段或采样代码：说明如何读 mcycle / minstret。

（2）计数器启停时机：需明确记录性能计数器采样的代码位置（例如在 timed region 前后）。

（3）数据同步机制说明：在 RV32 平台需说明如何处理高/低 32 位一致性读取。

5．分析要求

（1）核心行为解释：CPI/IPC 与基线微结构关系：解释 baseline CPI 如何与流水线中停顿或冒险行为相关（例如 load- use penalty、branch flush 等）。实例说明：选取典型 CoreMark 子工作负载（如循环、链表操作）讨论其在 baseline vs 优化版上的表现差异。

（2）优化方案对性能变化的归因：优化意图：简述为何该方向会减少某类停顿或提升某类指令吞吐。预期 vs 实际：对比预期性能变化（理论推导）与实验结果的吻合程度。统计验证：必要时附上对单条性能计数器指标的对比图或停顿类型计数。

七、思考题

1．为何 CoreMark 的得分不能单纯用总运行时间来评价处理器性能？请结合 CoreMark 的定义及其输出结构（Iterations/Sec、CRC 验证等）说明高得分意味着什么，以及使用不同编译器/flags 时可能造成什么误导。

2．解释 CPI 与 IPC 的含义及其计算方法。在你的实验结果中如何根据性能计数器数据（周期数和退休指令数）计算出 CPI 与 IPC，并说明这些指标如何反映流水线性能？

3．什么是流水线冒险（hazards）？分别从结构、数据、控制三个方面举例说明其对 CPI 的

影响。在你的 CoreMark 基线运行中，结合输出指标和指令执行特性，推测哪类冒险更可能成为性能瓶颈？

4．分支指令为什么对流水线性能影响显著？结合经典 RISC 流水线以及分支处理机制（不预测、延迟插槽或静态预测等），解释控制冒险如何影响流水线效率，并在你的实验 CoreMark 中分析分支行为对 CPI/IPC 的贡献。

5．在你实施的微结构优化中（任选一种：如 forwarding、分支预测等），请定量说明优化前后 CPI 和 IPC 的变化。数值上为什么会出现这种变化？请将改动与流水线中的具体冒险或停顿联系起来解释。

6．如果某个优化提高了 IPC 但在部分基准上却使得 CoreMark 分数提升有限，可能是什么原因？请结合 pipeline depth、分支行为和 memory latency 分析为什么 IPC 改善未必完全转化为得分提升。

7．实验中观察到的性能计数器数据出现异常（如 instret 大于预期或周期计数偏离预期）时，可能原因有哪些？请从计数器配置、stall 插入位置与 benchmark 运行方式等角度探讨可能原因，并提出验证方法。

八、附录

附录 A 微结构优化方向

1. 增加 Load-Use 数据旁路转发（Data Forwarding）

目标：减少 load → 使用 数据相关引入的停顿（stall）。

背景：Load-Use 相关是 in-order 流水线中最常见的数据冒险来源，往往在 load 后紧接使用所载入数据的指令中产生停顿。通过设计简单的 旁路（forwarding/bypass）路径，可以将数据从 LSU（Load Store Unit）直送给依赖它的执行单元而不必等完整写回寄存器堆。

预期改进：CPI 下降、IPC 上升，特别是在数据密集型循环 workload 上。

衡量标准：比较 baseline 与 modified 版本在加载密集 workload 或 CoreMark 子集（load 密集段）上的 CPI/IPC 改变。

2. 简单静态分支预测 + 分支目标缓冲（Static Branch/BTB）

目标：降低控制冒险导致的 pipeline flush 与停顿。

背景：条件分支跳转是流水线性能的主要灾难点之一，静态预测策略（如 “始终 not taken” 或小型 Branch Target Buffer）可以简单猜测下一条 PC，减少错误预测带来的 pipeline 重启开销。

预期改进：分支密集代码段 CPI 改善；CoreMark 中循环结构性能波动减小。

衡量标准：对比预测正确率与 flush 计数、整体 CPI/IPC 改变。

3. 延迟分支插槽（Branch Delay Slot）与编译器调度配合

目标：利用编译器/硬件协同减少分支停顿。

背景：有些 RISC ISA 设计允许在分支指令之后保留一个延迟执行指令（delay slot），将有用指令填入该 slot 可以有效利用 otherwise idle 周期。

预期改进：对分支后常见停顿位置填充有用指令，从而提升 IPC。

衡量标准：与 static 分支预测结合对比 CPI。

4. 轻量级多周期单元优化（Mul/Div Pipeline Anpassung）

目标：改善乘法/除法等多周期执行单元对流水线前段的阻塞。

背景：E203 核通常含有多周期计算单元（例如整数乘除），若该单元与 EX stage 无良好隔离，会阻塞后续指令发射或需要额外 stall。通过将这些计算单元单独 pipeline 或前推其部分计算结果，可减少对通用执行通路的阻塞。

预期改进：对含多次 multiply/div code 片段的程序（CoreMark 子项） CPI 改善。

衡量标准：对比多乘除工作负载的 CPI/IPC。

5. 管线冲突计数与可观测性增强（Performance Counter Expansion）

目标：改善性能分析精度，从而为进一步硬件优化提供更精准的量化依据。

背景：真实性能提升往往需要知道哪些冒险或停顿实际成为瓶颈。通过扩充简单的 停顿计

数器（stall counters：如 load- use stalls、branch flush stalls、structural stall counts 等），可以更精细地理解基线行为。

预期改进：虽然不直接提升 IPC，但可显著提高瓶颈定位准确性，从而为其他优化奠定计量基础。

衡量标准：提供更细粒度的 CPI 分解，从而更精准量化其它优化收益。

附录 B 每个方向的可衡量结果示例

为保证实验可量化、可对比，每个方向建议包含如下分析输出：

- Baseline vs Optimized 的 CPI、IPC 比较表（以 CoreMark 一组结果为样例）。
- 冒险停顿计数对比（如 load- use 停顿次数、branch flush 次数）。
- 对应优化前/后核心代码片段 instruction trace 示例。
- 对优化对 area/时序影响的简要定性分析（若涉及硬件改动）。

附录 C 参考资料

HBirdv2 E203

https://doc.nucleisys.com/hbirdv2/quick_start/